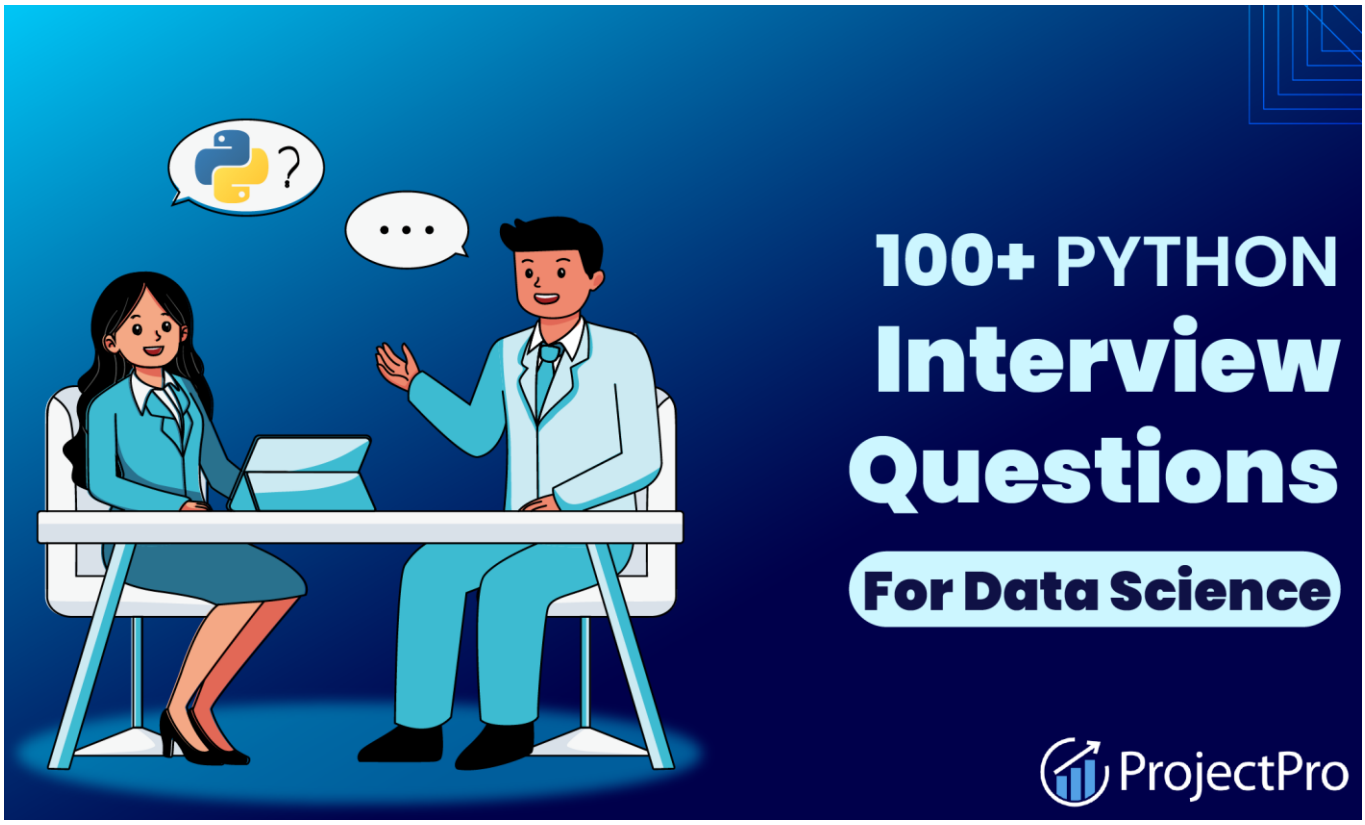
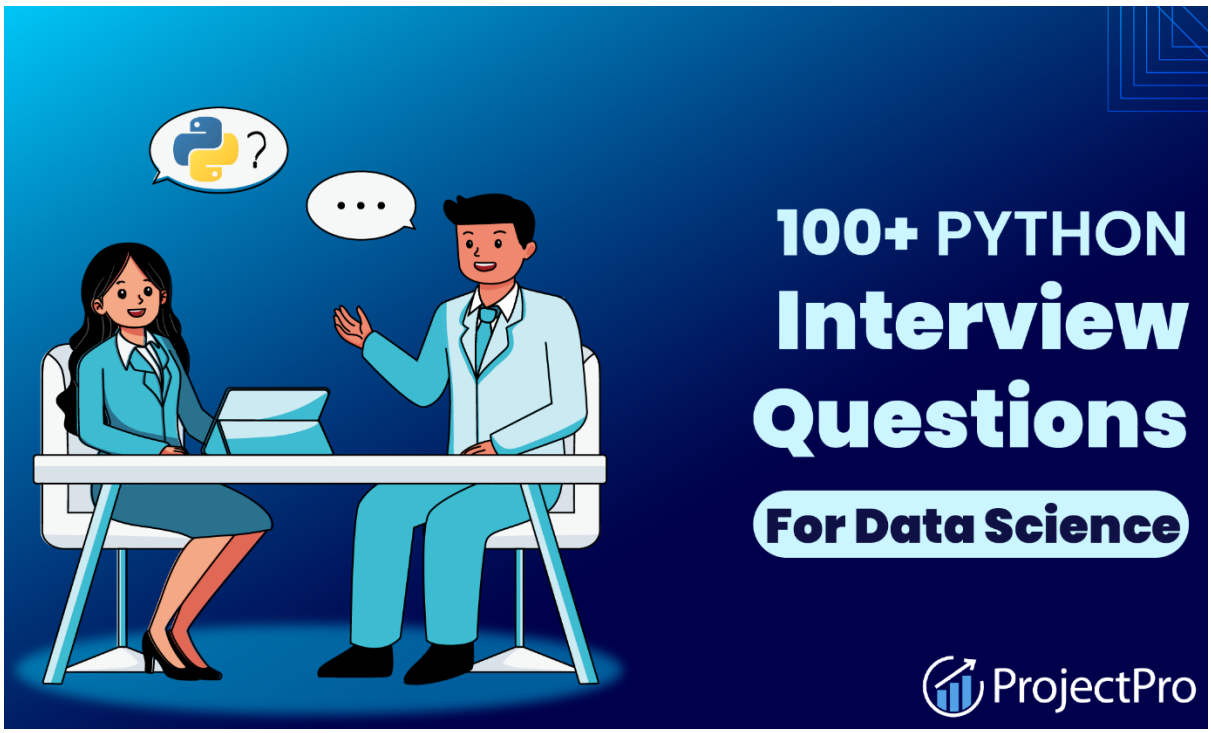

100+ PYTHON
Interview
Questions
For Data Science
ProjectPro

100+ Data Science in Python Interview Questions and Answers for 2025

Preparing for a Data Science interview in 2025? Python remains the go-to language for data analysis, machine learning, and AI, making it essential to master key concepts and problem-solving techniques. This blog compiles 100 must-know Python interview questions, covering topics like data manipulation, machine learning algorithms, and real-world coding challenges. No matter your experience level, these questions will help you sharpen your skills and ace your next interview. So, start practicing with detailed explanations and hands-on examples!



Top Data Science Python Interview Questions and Answers

Preparation is the key to bridging the gap between your skills and securing a data science/data analyst job. While interview structures may vary across companies, most follow a multi-stage process that assesses both technical expertise and problem-solving abilities.

The snapshot below (captured from an article) by Iain Brown, Head of Data Science at SAS, outlines the key stages of a typical data science interview, including technical assessments, project reviews, and behavioral evaluations. Understanding the expectations at each stage enables you to strategize your preparation, refine your problem-solving approach, and

effectively showcase your expertise. A structured preparation plan can make all the difference in standing out as a strong candidate.

Understanding the Data Science Interview Process

> Preparation is the bridge between having skills and landing the job

Data science interviews can vary widely depending on the organization, but they generally consist of several common stages:

1. **Phone Screen:** Typically conducted by HR to assess general fit.
2. **Technical Interview:** Involves coding challenges, statistical questions, and problem-solving tasks.
3. **Project Review:** Candidates may be asked to discuss past projects or take-home assignments.
4. **Behavioral Interview:** Focuses on understanding how a candidate's experience aligns with the team and company culture.
5. **Final Interview:** Often with higher management, focusing on the candidate's potential impact and alignment with company goals.

Understanding each stage's expectations is crucial for effective preparation.

Typical Distribution of Questions by Interview Type

In this blog, we have categorized the Top Data Science Python Interview Questions into different levels to help you prepare effectively. We cover:

- **Beginner-Level Questions-** Fundamental concepts, data manipulation, and exploratory analysis.
- **Intermediate Python Interview Questions-** Data visualization, feature engineering, and model building.
- **Advanced Python Interview Questions-** Deep learning, NLP, big data, and optimization techniques.
- **FAANG Interview Questions-** Real-world, high-difficulty problems asked by top tech companies.
- **Python Coding Interview Questions-** Algorithmic and coding-based questions to test problem-solving skills.

This structured approach ensures you cover everything from fundamentals to advanced problem-solving, making you well-prepared for any data science interview.

Basic Python Interview Questions and Answers for Beginners in Data Science

These beginner-friendly questions will help you build a strong foundation in Python programming, python data structures, essential mathematical and statistical operations for data manipulation, and data science libraries like NumPy, Pandas, and Matplotlib.

1) What is Python, and what are its key features?

Python is a high-level, interpreted programming language known for its simplicity and readability. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

Key Features of Python

1. Easy to Read and Write – The Python syntax is easy to understand.
2. Interpreted Language – Python executes code line by line, making debugging easier.
3. Dynamically Typed – Variables do not require explicit type declaration.

4. Extensive Libraries – Comes with a vast collection of built-in modules.
5. Cross-Platform – Python is platform-independent, meaning it runs on Windows, macOS, and Linux.
6. Garbage Collection – Python automatically manages memory using garbage collection.
7. Object-Oriented – Supports OOP concepts like encapsulation, inheritance, and polymorphism.

2) How can you build a simple logistic regression model in Python?

Logistic Regression is a supervised learning algorithm used for binary classification problems. It estimates the probability that an instance belongs to a particular class using the sigmoid function.

Example: Building a Logistic Regression Model in Python

We'll use the Iris dataset and classify whether a flower belongs to the "Setosa" species based on petal length and width.

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression

from sklearn.datasets import load_iris

from sklearn.metrics import accuracy_score

# Load dataset

iris = load_iris()

X = iris.data[:, :2]  # Selecting only two features for simplicity

y = (iris.target == 0).astype(int)  # Binary classification (Setosa or not)

# Split data into train and test sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train logistic regression model

model = LogisticRegression()

model.fit(X_train, y_train)

# Make predictions

y_pred = model.predict(X_test)

# Evaluate model

accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy:.2f}")
```

Key Steps in the Process

- Load the dataset – Use a sample dataset (Iris) with two features.
- Preprocess the data – Convert the target variable into binary form.
- Split the data – Divide into training and testing sets.
- Train the model – Fit a LogisticRegression model.
- Make predictions – Use the model to predict on the test data.

- Evaluate performance – Measure accuracy using accuracy_score.

This is a simple yet effective way to build a Logistic Regression classifier in Python

3) How can you train and interpret a linear regression model in SciKit-Learn?

Linear Regression is one of fundamental machine learning models used for predicting a continuous dependent variable based on one or more independent variables.

Steps to Train and Interpret a Linear Regression Model

We will use Scikit-Learn to build a regression model that predicts house prices based on features like square footage.

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score
```

```
# Sample dataset (House size vs. Price)

np.random.seed(42)

X = 2.5 * np.random.rand(100, 1)  # House size (in 1000 sq ft)

y = 50000 + (80000 * X) + (20000 * np.random.randn(100, 1)).flatten()  # House price with
noise

# Split data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Linear Regression Model

model = LinearRegression()

model.fit(X_train, y_train)

# Make predictions

y_pred = model.predict(X_test)

# Model interpretation

print(f"Intercept: {model.intercept_:.2f}")

print(f"Coefficient: {model.coef_[0]:.2f}")

# Model evaluation

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse:.2f}")

print(f"R² Score: {r2:.2f}")

# Visualization

plt.scatter(X_test, y_test, color='blue', label="Actual Prices")

plt.plot(X_test, y_pred, color='red', linewidth=2, label="Predicted Prices")
```

```
plt.xlabel("House Size (1000 sq ft)")

plt.ylabel("House Price ($)")

plt.legend()

plt.title("Linear Regression Model")

plt.show()
```

Key Takeaways from Model Interpretation

- Intercept (model.intercept_) – The expected house price when the house size is zero.
- Coefficient (model.coef_) – The amount by which house price changes per unit increase in house size.
- Mean Squared Error (MSE) – Measures how far predictions deviate from actual values (lower is better).
- R^2 Score (r2_score) – Shows how well the model explains the variation in house prices (closer to 1 is better).

This demonstrates how to train, interpret, and evaluate a Linear Regression model using Scikit-Learn!

4) Name a few libraries in Python used for Data Analysis and Scientific computations.

Python offers a rich ecosystem of libraries for data analysis and scientific computing, making it a popular choice for data science, machine learning, and research applications. Here are some essential libraries:

- NumPy – Provides support for multi-dimensional arrays, mathematical functions, and linear algebra operations.
- Pandas – Used for data manipulation and analysis, offering DataFrame structures similar to SQL tables.
- Matplotlib – A plotting library for visualizing data through charts, graphs, and histograms.
- Seaborn – Built on Matplotlib, it provides advanced statistical visualizations.

- Scikit-Learn – The go-to library for machine learning, including regression, classification, and clustering.
- SciPy – Offers scientific computing functions, including optimization, signal processing, and statistics.

These libraries form the core toolkit for data scientists, enabling efficient analysis, visualization, and model building.

5) Which library would you prefer for plotting in the Python language: Seaborn or Matplotlib?

Matplotlib is the python library used for plotting but it needs a lot of fine-tuning to ensure that the plots look shiny. Seaborn helps data scientists create statistically and aesthetically appealing meaningful plots. The answer to this question varies based on the requirements for plotting data.

6) What is the main difference between a Pandas series and a single-column DataFrame in Python?

A Pandas Series is a one-dimensional labeled array, whereas a single-column DataFrame is a two-dimensional structure that contains one Series but with an additional index.

Example

```
import pandas as pd

# Creating a Pandas Series

series = pd.Series([10, 20, 30])

print(type(series))  # Output: <class 'pandas.core.series.Series'>

# Creating a single-column DataFrame

df = pd.DataFrame({'A': [10, 20, 30]})

print(type(df))  # Output: <class 'pandas.core.frame.DataFrame'>
```

7) Write code to sort a DataFrame in Python in descending order.

You can use the .sort_values() method to sort a DataFrame.

Example

```
import pandas as pd

# Creating a sample DataFrame

df = pd.DataFrame({'A': [3, 1, 4, 2], 'B': [8, 6, 5, 9]})

# Sorting by column 'A' in descending order

df_sorted = df.sort_values(by='A', ascending=False)

print(df_sorted)
```

8) How can you handle duplicate values in a dataset for a variable in Python?

You can identify, remove, or replace duplicate values using Pandas functions.

Example

```
import pandas as pd

# Creating a sample DataFrame

df = pd.DataFrame({'A': [1, 2, 2, 3, 3, 3], 'B': [4, 5, 5, 6, 6, 7]})

# Removing duplicates

df_cleaned = df.drop_duplicates()

print(df_cleaned)
```

- .drop_duplicates() removes duplicate rows.
- .duplicated() returns a Boolean Series indicating duplicates.

9) Which Random Forest parameters can be tuned to enhance the predictive power of the model?

Some key hyperparameters to tune in Random Forest:

- n_estimators – Number of trees in the forest
- max_depth – Maximum depth of each tree
- min_samples_split – Minimum samples required to split a node
- min_samples_leaf – Minimum samples required to be at a leaf node
- max_features – Number of features to consider at each split
- bootstrap – Whether to use bootstrap samples
- criterion – ‘gini’ or ‘entropy’ for classification, ‘mse’ or ‘mae’ for regression

10) Which method in pandas.tools.plotting is used to create a scatter plot matrix?

The method used is scatter_matrix() from pandas.plotting.

Example

```
from pandas.plotting import scatter_matrix

import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

scatter_matrix(df)
```

11) How can you check if a data set or time series is Random?

You can use a lag plot to check randomness. If the plot shows no pattern, the dataset is random.

Example

```
import pandas as pd

import matplotlib.pyplot as plt
```

```
from pandas.plotting import lag_plot

df = pd.Series([1, 3, 2, 4, 3, 5, 6, 7])

lag_plot(df)

plt.show()
```

12) Can we create a DataFrame with multiple data types in Python? If yes, how can you do it?

Yes, Pandas DataFrames allow multiple data types.

Example

```
import pandas as pd

df = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30], 'Salary': [50000.5, 70000.8]})

print(df.dtypes)  # Output: Name - object, Age - int64, Salary - float64
```

13) Is it possible to plot histogram in Pandas without calling Matplotlib? If yes, then write the code to plot the histogram?

Yes, Pandas integrates directly with Matplotlib for plotting.

Example

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 2, 3, 3, 3, 4, 5]})

df['A'].hist()
```

14) What are the possible ways to load an array from a text data file in Python? How can the efficiency of the code to load data file be improved?

You can use numpy.loadtxt() or pandas.read_csv().

Example

```
import numpy as np

data = np.loadtxt('data.txt')
```

15) Which is the standard data missing marker used in Pandas?

The standard marker is NaN (Not a Number).

16) Why you should use NumPy arrays instead of nested Python lists?

- Faster computations – NumPy arrays are optimized for performance.
- Less memory consumption – Efficiently stored in contiguous memory.
- Vectorized operations – Faster arithmetic without explicit loops.

17) List down some evaluation metrics for regression problems.

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score (Coefficient of Determination)

18) Which Python library would you prefer to use for Data Munging?

Pandas is the most widely used library for data cleaning and preprocessing.

19) Write the code to sort an array in NumPy by the nth column?

Using the argsort() Python function, the solution can be achieved. If there is an array X and you would like to sort the nth column then code for this will be x[x [: n-1].argsort ()]

Example

```
import numpy as np

arr = np.array([[1, 3], [2, 2], [3, 1]])

sorted_arr = arr[arr[:, 1].argsort()]

print(sorted_arr)
```

20) How are NumPy and SciPy related?

- NumPy provides array operations and mathematical functions.
- SciPy builds on NumPy, adding scientific computing functions like optimization, integration, and signal processing.

21) Which python library is built on top of matplotlib and Pandas to ease data plotting?

Seaborn

22) Which plot will you use to assess the uncertainty of a statistic?

The Bootstrap plot is used to evaluate statistical uncertainty.

23) What are some features of Pandas that you like or dislike?

Pros

- Easy data manipulation
- Powerful groupby and aggregation functions
- Seamless integration with NumPy and Matplotlib

Cons

- Can be slow for very large datasets
- Memory-intensive compared to alternatives like Dask

24) Which scientific libraries in SciPy have you worked with in your project?

- scipy.optimize – Optimization algorithms
- scipy.stats – Statistical functions
- scipy.linalg – Linear algebra functions

25) What is pylab?

A package that combines NumPy, SciPy and Matplotlib into a single namespace.

26) Which python library is used for Machine Learning?

Scikit-Learn is the most commonly used library for Machine Learning.

27) What is the preferred method to check for an empty array in NumPy?

Use size to check if an array is empty.

Example

```
import numpy as np

arr = np.array([])

print(arr.size == 0)  # Output: True
```

Intermediate Python Interview Questions and Answers in Data Science

After mastering the basics of Python for data science, it's essential to step up and explore intermediate-level concepts. These interview questions are designed to test your ability to work with real-world datasets, write optimized code, and implement machine learning models effectively.

28) What is a KeyError in Python, and how do you handle it?

A KeyError occurs when attempting to access a key that does not exist in a dictionary. This happens because Python expects all requested keys to be present in the dictionary, and when one is missing, it raises an exception.

Ways to Handle KeyError

- Use .get(): The function returns None or a default value instead of an error.
- Try-Except Block: Catches the error and provides a fallback.
- Check Key Existence: Use the if key in the Python dictionary before accessing it.

Example

```
data = {"name": "Alice", "age": 25}

# Using .get()

print(data.get("city", "Not found"))  # Output: Not found

# Using try-except

try:

    print(data["city"])

except KeyError:

    print("Key does not exist!")  # Output: Key does not exist!
```

29) How does Python manage memory, and what role does garbage collection play?

Python automatically manages memory using a private heap, where all objects and data structures reside. The memory manager handles allocation, while garbage collection (GC) removes objects that are no longer needed, freeing up memory.

Garbage Collection in Python

- Uses reference counting to track objects.
- Has a cyclic garbage collector to detect and remove circular references.
- The gc module allows manual control over garbage collection.

Example

```
import gc

gc.collect()  # Forces garbage collection
```

This process ensures efficient memory usage, preventing leaks and optimizing performance.

30) What is the difference between shallow copy and deep copy in Python?

Both shallow and deep copies create duplicates of objects, but they differ in how they handle nested structures.

- Shallow Copy: Creates a new object but maintains references to mutable objects inside it.
- Deep Copy: Recursively duplicates everything, creating a fully independent copy.

Example

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)

deep = copy.deepcopy(original)

shallow[0][0] = 99  # Modifies both original and shallow copy

print(original)  # [[99, 2], [3, 4]]

deep[0][0] = 42  # Modifies only the deep copy

print(original)  # [[99, 2], [3, 4]]
```

Use a shallow copy for flat structures and a deep copy for complex nested objects.

31) How does Python's collections module simplify tasks?

The collections module provides specialized data structures that improve efficiency and simplify common tasks.

- Counter: Counts occurrences of elements in an iterable.
- defaultdict: Avoids KeyError by providing a default value for missing keys.
- deque: Optimized list-like structure for fast appends and pops.
- OrderedDict: Preserves insertion order (important in older Python versions).

Example

```
from collections import Counter

data = ['a', 'b', 'c', 'a', 'b', 'a']

count = Counter(data)

print(count)  # Output: Counter({'a': 3, 'b': 2, 'c': 1})
```

These structures optimize performance for specific use cases, making Python more powerful.

32) What is monkey patching in Python?

Monkey patching is a dynamic technique that allows modifying or extending existing classes or modules at runtime without altering their source code.

Example

```
class Animal:

    def sound(self):

        print("Original sound")

def new_sound(self):

    print("Patched sound!")

# Monkey patching

Animal.sound = new_sound

a = Animal()

a.sound()  # Output: Patched sound!
```

It’s useful for hotfixes and testing but should be used with caution to avoid unexpected behavior.

33) What is the purpose of Python's with statement?

The with statement simplifies resource management (e.g., file handling) by ensuring proper cleanup after use.

Without with:

```
file = open("example.txt", "w")

file.write("Hello, world!")

file.close()
```

With with:

```
with open("example.txt", "w") as file:

    file.write("Hello, world!")  # Auto-closes file
```

This ensures the file closes automatically, even if an error occurs.

34) Why use else in a try-except block?

The else block in try-except executes only if no exception occurs, making code cleaner and more structured.

Example

```
try:

    num = int(input("Enter a number: "))

    print("Valid number:", num)

except ValueError:

    print("Invalid input!")

else:

    print("Number processing completed.")
```

If input is valid, else ensures additional logic runs only when no errors occur.

35) What are decorators in Python?

Decorators are functions that modify other functions without changing their structure. They are often used for logging, authentication, or timing functions.

Example

```
def decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper
@decorator
def greet():
    print("Hello!")
greet()
```

Output:

```
pgsql
Before function call
Hello!
After function call
```

Decorators enable clean, reusable, and modular code.

36) What are context managers in Python?

Context managers manage resources efficiently, ensuring they are properly acquired and released. They are primarily used with the with statement.

Example (Custom Context Manager):

```
class FileManager:

    def __init__(self, filename, mode):

        self.filename = filename

        self.mode = mode

    def __enter__(self):

        self.file = open(self.filename, self.mode)

        return self.file

    def __exit__(self, exc_type, exc_value, traceback):

        self.file.close()

with FileManager("test.txt", "w") as f:

    f.write("Hello, world!")  # File auto-closes
```

This ensures files close automatically, even if an error occurs.

37) What are metaclasses in Python?

Metaclasses define how classes are created in Python, acting as "classes of classes." They allow customization of class behavior at the time of creation.

Example

```
class Meta(type):

    def __new__(cls, name, bases, dct):

        print(f"Creating class: {name}")
```

```
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):

    pass
```

Output

Creating class: MyClass

Metaclasses are useful for enforcing coding standards or dynamically modifying class behavior.

38) What are the advantages of NumPy over regular Python lists?

NumPy offers significant improvements over standard Python lists, particularly in performance and functionality:

- Memory Efficiency: NumPy arrays use less memory because they store data in a contiguous block, unlike Python lists that store references.
- Faster Computation: NumPy operations run significantly faster because they leverage optimized C-based functions.
- Vectorized Operations: Supports element-wise operations without explicit loops.
- Built-in Mathematical Functions: Includes efficient operations for linear algebra, statistics, and more.

Example

```
import numpy as np

arr = np.array([1, 2, 3])

print(arr * 2)  # Output: [2 4 6] (element-wise multiplication)
```

NumPy is ideal for handling large datasets and performing scientific computations.

39) What is the difference between merge(), join(), and concat() in pandas?

These functions combine DataFrames but differ in approach:

- merge(): Combines two DataFrames based on a common column, like SQL joins.
- join(): Merges DataFrames based on index instead of columns.
- concat(): Stacks multiple DataFrames along rows or columns.

Example

```
import pandas as pd

df1 = pd.DataFrame({'ID': [1, 2], 'Name': ['Alice', 'Bob']})

df2 = pd.DataFrame({'ID': [1, 2], 'Score': [90, 85]})

# Merge using a common column

merged_df = pd.merge(df1, df2, on='ID')

print(merged_df)
```

Understanding when to use each ensures efficient data manipulation.

40) How do you identify and handle missing values in pandas?

Missing values can be identified using:

```
df.isnull().sum()  # Returns count of missing values per column
```

Ways to replace missing values:

- Drop rows/columns: Use dropna() (not recommended for large datasets).
- Fill missing values: Use fillna() with a constant, mean, or method like forward-fill (ffill).
- Interpolate: Use interpolate() for numerical trends.

Example

```
df.fillna(value=-999)  # Replaces NaN values with -999

df.interpolate(method='linear')  # Estimates missing values
```

Handling missing data properly ensures accurate analysis.

41) What Python libraries are used for Data Visualization?

Python offers several powerful libraries for visualization:

- Matplotlib: Basic plotting for static charts like line, bar, and scatter plots.
- Seaborn: Built on Matplotlib, ideal for statistical visualizations.
- Plotly: Interactive plots with zooming and hovering.
- Bokeh: Used for interactive and web-based visualizations.

Example using Seaborn

```
import seaborn as sns

import matplotlib.pyplot as plt

sns.set_theme()

data = sns.load_dataset("tips")

sns.scatterplot(x="total_bill", y="tip", data=data)

plt.show()
```

Choosing the right library depends on the level of interactivity and complexity needed.

42) How would you normalize or standardize a dataset in Python?

- Normalization scales values between 0 and 1.
- Standardization transforms data to have a mean of 0 and a standard deviation of 1.

Example using scikit-learn

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

import numpy as np

data = np.array([[1, 2], [3, 4], [5, 6]])

# Normalize

normalizer = MinMaxScaler()

normalized = normalizer.fit_transform(data)

# Standardize

scaler = StandardScaler()

standardized = scaler.fit_transform(data)
```

Choosing between normalization and standardization depends on the dataset and model requirements.

43) How can you replace spaces in a string with a given character in Python?

To replace spaces in a string with another character, use replace() or list comprehension.

Example using replace()

```
text = "Data Science is awesome"

print(text.replace(" ", "_"))  # Output: Data_Science_is_awesome
```

Example using list comprehension

```
def replace_space(text, char):

    return "".join([char if c == " " else c for c in text])

print(replace_space("Hello World", "-"))  # Output: Hello-World
```

This is useful for formatting filenames, URLs, and data preprocessing.

Advanced Python Data Science Interview Questions and Answers

Go through the following python interview questions for data science that are slightly advanced. These python data science interview questions might be difficult for you to answer but it is important that you prepare for these python interview questions as well before going for your interview.

44) What is the Global Interpreter Lock (GIL) in Python, and why is it important?

The Global Interpreter Lock (GIL) in CPython ensures that only one thread executes Python bytecode at a time, limiting true multi-threading. It prevents race conditions in memory management but hinders CPU-bound tasks from utilizing multiple cores efficiently. While I/O-bound tasks (e.g., file handling, web requests) perform well with multi-threading, CPU-bound tasks (e.g., data processing, machine learning) benefit more from multiprocessing, which bypasses the GIL. Alternatives like PyPy, Cython, or Numba can mitigate its impact. The GIL simplifies memory management but is a bottleneck for performance-heavy applications requiring parallel execution.

45) What Are Some Essential Searching and Graph Traversal Algorithms in Python?

Python offers a range of efficient algorithms for searching and navigating data structures, each tailored to different needs:

- Linear Search: The simplest searching algorithm that scans through a list one item at a time until the target is found. It works on unsorted data but has an O(n) time complexity, making it inefficient for large datasets.
- Binary Search: When working with sorted lists, binary search is the fastest way to locate an element. It repeatedly divides the search range in half, achieving an O(log n) time complexity.
- AVL Tree: This self-balancing binary search tree ensures optimal search times, even with frequent insertions or deletions, by keeping the tree height balanced at O(log n).
- Breadth-First Search (BFS): Perfect for exploring graphs level by level, BFS is commonly used for shortest path discovery in unweighted graphs, social networks, and recommendation systems.

- Depth-First Search (DFS): DFS goess deep into branches before backtracking, making it great for maze-solving, tree traversal, and cycle detection in graphs.
- A Algorithm*: A hybrid of BFS and heuristics, A* efficiently finds the shortest path by prioritizing promising routes. It's widely used in AI, robotics, and game development.

Each of these algorithms serves a unique purpose, ensuring optimal performance for different data structures and real-world applications.

46) What are universal functions for n-dimensional arrays?

Universal functions are the functions that perform mathematical operations on each element of an n-dimensional array.
Example: np.sqrt() and np.exp() evaluate square root and exponential of each element of an array respectively.

47) List a few statistical methods available for a NumPy array.

np.means(), np.cumsum(), np.sum(),

48) What are boolean arrays? Write a code to create a boolean array using the NumPy library.

A Boolean array in NumPy is an array where each element is either True or False. It is commonly used for filtering data, masking values, and performing conditional operations efficiently.

Example: Creating a Boolean Array in NumPy

You can create a Boolean array using conditions on NumPy arrays.

```
import numpy as np

# Creating a NumPy array

arr = np.array([10, 20, 30, 40, 50])

# Creating a Boolean array where elements > 25 are True

bool_arr = arr > 25

print(bool_arr)
```

Output: [False False True True True]

Creating a Boolean Array Directly

You can also create a Boolean array explicitly:

bool_array = np.array([True, False, True, False])

print(bool_array)

Output: [True False True False]

49) What is Fancy Indexing?

In NumPy, one can use an integer list to describe the indexing of NumPy arrays. For example, Array[[2,1,0,3]] for an array of dimensions 4x4 will print the rows in the order specified by the list.

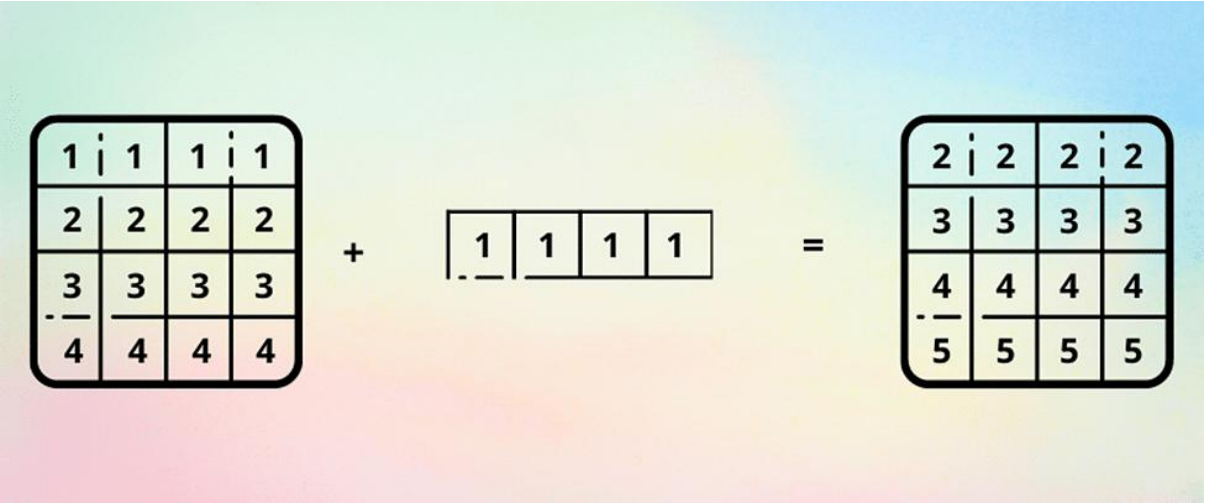
50) What is NaT in Python's Pandas library?

NaT stands for Not a Time. It is the NA value for timestamp data

51) What is Broadcasting for NumPy arrays?

Broadcasting is a technique that specifies how arithmetic calculations are performed between arrays of different dimensions.

This can be represented by the following image:



52) What is the necessary condition for broadcasting two arrays?

The two arrays must satisfy either of the following conditions:

- For each dimension starting from the end, the axis lengths should be equal.
- Either of the matrices should be one-dimensional

53) What is PEP for Python?

PEP stands for Python Enhancement Proposal. It is a document that provides information related to new features of Python, its processes or environments.

54) What do you mean by overfitting a dataset?

Overfitting a dataset means our model is fitting the training dataset so well that it performs poorly on the test dataset. One of the key reasons for overfitting could be that the model has learned the noise in the dataset.

55) What do you mean by underfitting a dataset?

Underfitting a dataset means our model is fitting the training dataset poorly. It usually occurs when we don't fine-tune the parameters of a model and keep looking for alternatives.

56) What is the difference between a test set and a validation set?

For unsupervised learning, we use a validation set for selecting a model based on the estimated prediction error. On the other hand, we use a test set to assess the accuracy of the finally chosen model.

57) What is F1-score for a binary classifier? Which library in Python contains this metric?

The F1-score is a combination of precision and recall that represents the harmonic mean of the two quantities. It is given by the formula

$$F_1 = 2 \times \frac{Precision\ .\ Recall}{Precision + Recall}$$

58) Write a function for f1_score that takes True Positive, False Positive, True Negative, and False Negative as input and outputs f1_score.

```
def f1_score(tp, fp, fn, tn):
  p =  tp / (tp + fp)
  r = tp / (tp + fn)
  return 2 * p * r / (p + r)
```

59) Using sklearn library, how will you implement ridge regression?

```
>>> from sklearn import linear_model
>>>reg = linear_model.LinearRegression()
>>> reg = linear_model.Ridge(alpha=0.5)
>>> reg.fit(sample_dataset)
```

60) Using sklearn library, how will you implement lasso regression?

```
>>> from sklearn import linear_model
>>>reg = linear_model.LinearRegression()
>>> reg = linear_model.Lasso(alpha=0.4)
>>> reg.fit(sample_dataset)
```

61) How is correlation a better metric than covariance?

Covariance is a metric that reflects how two variables (a and b) vary from their respective average values (Ä• and Æ€). It is given by

$$cov_{a,b} = \frac{\sum(a_i - \bar{a})(b_i - \bar{b})}{n-1}$$

Where N is the number of data points.

Correlation is a metric that takes into account the standard deviations of the variables (a and b). Mathematically, it is defined as,

$$cor_{a,b} = \frac{\sum(a_i - \bar{a})(b_i - \bar{b})}{\sqrt{(a_i - \bar{a})^2(b_i - \bar{b})^2}}$$

Covariance for variables that have large deviations from the mean would become large but the variables could still be related to each other. Correlation is thus a better metric than covariance for it divides out the standard deviations of the variables.

62) What are confounding factors?

Cofounding factors are the variables that relate to both dependent and independent variables. It cannot be picked through the evaluation of correlations.

63) What is namespace in Python?

A namespace is a collection of names that are created when we start running a Python interpreter and continue to exist till the interpreter is running.

64) What is try-except-finally in Python?

If we want to write a code in Python, and we are not sure whether it is error-free or not, then we can use try-except-finally in Python.

We use ‘try’ to test a block of code for the error.

We use ‘except’ to handle the error.

We use ‘finally’ to execute the remaining code irrespective of the result of try and except blocks.

Example,

```
>>>try:
   print(a)
>>>except:
   print("Something is not right! ")
```

```
>>>finally:

  print("The 'try except block' is over")
```

65) What is the difference between append() and extend() functions in Python?

1. append(): Append() is a function in Python that adds the element received at the input to the end of the list. It increments the size of the list by one.

Example

```
>>>List1 = ['I', 'love']

>>>List1.append(['ProjectPro', 'and', 'Dezyre'])

>>>print(List1)
```

Output

```
['I', 'love', ['ProjectPro', 'and', 'Dezyre'] ]
```

2. extend(): Extend() is a function in Python that first iterates over each element of the input and then adds each element to the end of the list.

Example

```
>>>List1 = ['I', 'love']

>>>List1.extend(['ProjectPro', 'and', 'Dezyre'])

>>>print(List1)
```

Output

```
['I', 'love', 'ProjectPro', 'and', 'Dezyre' ]
```

66) What is the use of enumerate() function?

Enumerate() is a function in Python that assigns a counting label to each element of the iterable object and returns it in the form of an enumerate object as output.

Example

```
>>>List1 = ["eat","sleep","ProjectPro"]

>>>String1 = "Repeat"

>>>

>>># Enumerate objects

>>>object1 = enumerate(List1)

>>>object2 = enumerate(String1)

>>>print ("Return type:",type(object1))

>>>print (list(enumerate(List1)))

>>>

>>># Start index to 2 instead pf 0

>>>print (list(enumerate(String1,2)))
```

Output

```
Return type: < type 'enumerate' >

[(0, 'eat'), (1, 'sleep'), (2, 'ProjectPro')]

[(2, 'R'), (3, 'e'), (4, 'p'), (5, 'e'), (6,’a’),(7,’t’)]
```

67) What are the mutable and immutable built-in data types in Python? Provide examples.

Immutable Data Types (Cannot be changed after creation)	**Mutable Data Types (Can be modified)**
• Strings • Numbers (int, float, complex) • Tuples • Frozen Sets	• Lists • Dictionaries • Sets

Example

```
# Immutable Example

immutable_str = "Hello"

# immutable_str[0] = "h"  # This will raise an error

immutable_tuple = (1, 2, 3)

# immutable_tuple[0] = 10  # This will raise an error

# Mutable Example

mutable_list = [1, 2, 3]

mutable_list[0] = 10  # Allowed

print(mutable_list)  # Output: [10, 2, 3]

mutable_dict = {"name": "Alice"}

mutable_dict["age"] = 25  # Allowed

print(mutable_dict)  # Output: {'name': 'Alice', 'age': 25}
```

68) What is negative indexing in Python?

Most programming languages did not allow negative indexing and Python is one of those rare languages which supports that. Negative indexing means that one can use negative numbers to access the elements of an array. But, the key point to remember is that the index -1 represents the last element of the array, -2 represents the second last element of the array and so on. Thus, in negative indexing, the counting starts from where the array ends.

For Example

```
>>> a = “ProjectPro blogs are fun to read!”

>>>print(a[-5:-1])
```

Output

```
read
```

Python Data Science Coding Interview Questions with Solution Code

Below are some essential Python interview questions, designed to test problem-solving skills and understanding of core concepts.

69) How do you find the intersection of two lists in Python?

To determine common elements between two lists, you can use Python’s set operations.

Example

```
def list_intersection(lst1, lst2):

    return list(set(lst1) & set(lst2))

# Example usage

X = [1, 5, 9, 0]

Y = [3, 0, 2, 9]

print(list_intersection(X, Y))  # Output: [9, 0]
```

This method efficiently finds common values using O(n) time complexity.

70) How do you find duplicate elements in an array?

A simple approach is to track elements using a set.

Example

```
def find_duplicates(arr):
    seen = set()
    duplicates = set()
    for num in arr:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)
    return list(duplicates)
print(find_duplicates([1, 2, 3, 1, 3, 6, 5]))  # Output: [1, 3]
```

This approach ensures O(n) time complexity.

71) How do you find the maximum product of three numbers in an array?

To maximize the product, consider the largest three numbers or two smallest (negative) and the largest number.

Example

```
import heapq
def max_product_of_three(nums):
    largest = heapq.nlargest(3, nums)
    smallest = heapq.nsmallest(2, nums)
    return max(largest[0] * largest[1] * largest[2], smallest[0] * smallest[1] * largest[0])
```

```
print(max_product_of_three([-10, -3, 5, 6, 7]))  # Output: 420
```

72) How do you find the sum of the largest contiguous subarray?

Use Kadane's Algorithm to efficiently find the maximum subarray sum.

Example

```
def max_subarray_sum(arr):
    max_sum = float('-inf')
    current_sum = 0
    for num in arr:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum if max_sum > 0 else 0
print(max_subarray_sum([0, -1, -5, -2, 3, 14]))  # Output: 17
```

73) What is the difference between lists and tuples in Python?

Feature	**Lists ([])**	**Tuples (())**
Mutability	Mutable (modifiable)	Immutable (cannot change)
Performance	Slower, as memory allocation is dynamic	Faster, since memory is preallocated
Methods	More built-in methods	Fewer built-in methods
Use Case	Suitable for dynamic collections	Ideal for fixed, ordered data

74) How do you compute the Euclidean distance between two lists?

The Euclidean distance between two points (x1, y1) and (x2, y2) is given by:

d=(x2−x1)2+(y2−y1)2d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}d=(x2−x1)2+(y2−y1)2

Example

```
import numpy as np

def euclidean_distance(a, b):

    return np.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

print(euclidean_distance([1, 2, 3], [4, 5, 6]))  # Output: 5.196
```

75) How do you generate all k-combinations of numbers from 1 to n?

```
from itertools import combinations

def generate_combinations(n, k):

    return list(combinations(range(1, n + 1), k))

print(generate_combinations(3, 2))  # Output: [(1,2), (1,3), (2,3)]
```

76) How do you generate and plot samples from a normal distribution?

```
import numpy as np

import matplotlib.pyplot as plt

N = 1000  # Number of samples

samples = np.random.randn(N)

plt.hist(samples, bins=30, alpha=0.7, color='blue', edgecolor='black')

plt.show()
```

77) Difference between apply() and applymap() in pandas?

Function	Works On	Scope
apply()	DataFrames & Series	Row/column-wise transformations
applymap()	Only DataFrames	Element-wise transformations

Example

```
import pandas as pd

df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])

# Using apply()

df['A'] = df['A'].apply(lambda x: x * 2)

# Using applymap()

df = df.applymap(lambda x: x * 2)
```

78). How do you find the first recurring character in a string?

```
def first_recurring_char(s):

    seen = set()

    for char in s:

        if char in seen:

            return char

        seen.add(char)

    return None

print(first_recurring_char("pythoninterviewquestion"))  # Output: 'n'
```

79) How do you calculate the factorial of a number using recursion?

```
def factorial(n):
    if n < 0:
        return -1  # Invalid input
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
print(factorial(5))  # Output: 120
```

80) How do you find the longest increasing path in a matrix?

```
from functools import lru_cache
def longest_increasing_path(matrix):
    rows, cols = len(matrix), len(matrix[0])
    @lru_cache(None)
    def dfs(r, c, prev):
        if not (0 <= r < rows and 0 <= c < cols) or matrix[r][c] <= prev:
            return 0
        val = matrix[r][c]
        return 1 + max(dfs(r+1, c, val), dfs(r-1, c, val),
                       dfs(r, c+1, val), dfs(r, c-1, val))
    return max(dfs(r, c, -1) for r in range(rows) for c in range(cols))
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(longest_increasing_path(matrix))  # Output: 5
```

81) What is Flask and why is it useful?

Flask is a lightweight web framework for Python that allows developers to create web applications with minimal boilerplate.

Benefits of Flask

- Lightweight & Scalable: Ideal for small to large applications.
- Flexibility: Allows customization without forcing a strict structure.
- Built-in Development Server: Easy testing without external tools.
- Supports Extensions: Can integrate with databases, authentication, and APIs.

Example of a Simple Flask App

```
from flask import Flask

app = Flask(__name__)

@app.route('/')

def home():

    return "Hello, Flask!"

if __name__ == '__main__':

    app.run(debug=True)
```

82) How do you check if a number is a perfect square in Python?

A perfect square has an integer square root.

Example

```
def is_perfect_square(num):

    sqrt = int(num ** 0.5)

    return sqrt * sqrt == num
```

```
print(is_perfect_square(25))  # Output: True

print(is_perfect_square(26))  # Output: False
```

This method is efficient and avoids unnecessary calculations.

83) How do you count trailing zeros in a factorial (n!)?

Trailing zeros in n! come from factors of 10 (2 × 5). Since multiples of 2 are more frequent, count multiples of 5 in n!.

Example

```
def trailing_zeros(n):
    count = 0
    while n >= 5:
        n //= 5
        count += n
    return count

print(trailing_zeros(10))  # Output: 2

print(trailing_zeros(25))  # Output: 6
```

This runs in O(log n) time, making it efficient.

84) How do you check if a string can be segmented into dictionary words?

This problem involves dynamic programming.

Example

```
def can_segment(s, dictionary):
    for i in range(1, len(s) + 1):
        first, second = s[:i], s[i:]
        if first in dictionary and (not second or can_segment(second, dictionary)):
```

```
            return True
    return False
dictionary = {"data", "camp", "science"}
print(can_segment("datacamp", dictionary))  # Output: True
```

This is useful in NLP and spell-checking.

85) How do you remove duplicates from a sorted list?

Example using two-pointer technique

```
def remove_duplicates(arr):
    insert_idx = 1
    for i in range(1, len(arr)):
        if arr[i] != arr[i - 1]:
            arr[insert_idx] = arr[i]
            insert_idx += 1
    return insert_idx
arr = [1, 1, 2, 2, 3, 4, 4]
print(remove_duplicates(arr))  # Output: 4
```

This modifies the list in-place for efficiency.

86) How do you find a missing number in a range?

Using the sum formula:

```
def find_missing(nums, n):
    expected_sum = n * (n + 1) // 2
    actual_sum = sum(nums)
    return expected_sum - actual_sum
```

```
print(find_missing([1, 2, 4, 5, 6], 6))  # Output: 3
```

This approach runs in O(1) space and O(n) time.

87) How do you check if a string is a palindrome?

```
def is_palindrome(s):
    s = ''.join(c.lower() for c in s if c.isalnum())  # Remove non-alphanumerics
    return s == s[::-1]
print(is_palindrome("A man, a plan, a canal: Panama"))  # Output: True
print(is_palindrome("hello"))  # Output: False
```

This method is case-insensitive and removes special characters.

88) How can you copy objects in Python?

The functions used to copy objects in Python are-

1) Copy.copy () for shallow copy

2) Copy.deepcopy () for deep copy

However, it is not possible to copy all objects in Python using these functions. For instance, dictionaries have a separate copy method whereas sequences in Python have to be copied by 'Slicing'.

89) How does a child class inherit from a parent class in Python?

In Python, a child class inherits attributes and methods from a parent class using inheritance. This allows code reuse and the extension of functionality. The parent class is passed as an argument when defining the child class.

```
# Parent class
class Animal:
    def __init__(self, name):
        self.name = name
```

```
    def speak(self):
        return "Some generic sound"
# Child class inheriting from Animal
class Dog(Animal):
    def speak(self):
        return "Bark"
# Creating objects
dog = Dog("Buddy")
print(dog.name)   # Output: Buddy
print(dog.speak())  # Output: Bark
```

In this example, the Dog class inherits the name attribute from the Animal class but overrides the speak() method with a custom implementation.

90) How will you transpose a NumPy array?

Transposing a NumPy array means flipping its rows and columns. You can achieve this using .T or numpy.transpose().

Example: Using .T Attribute

```
import numpy as np
# Creating a 2D NumPy array
arr = np.array([[1, 2, 3], [4, 5, 6]])
# Transposing the array
transposed_arr = arr.T
print(transposed_arr)
```

Output

```
[[1 4]
 [2 5]
 [3 6]]
```

Example: Using numpy.transpose()

```
transposed_arr2 = np.transpose(arr)

print(transposed_arr2)
```

Both methods swap the rows with columns, making it useful in linear algebra, machine learning, and data processing

91) What is the difference between tuples and lists in Python?

Tuples can be used as keys for dictionaries i.e. they can be hashed. Lists are mutable whereas tuples are immutable - they cannot be changed. Tuples should be used when the order of elements in a sequence matters. For example, a set of actions that need to be executed in sequence, geographic locations or list of points on a specific route.

```
# List Example (Mutable)

my_list = [1, 2, 3]

my_list.append(4)  # Allowed

print(my_list)  # Output: [1, 2, 3, 4]

# Tuple Example (Immutable)

my_tuple = (1, 2, 3)

# my_tuple.append(4)  # This will raise an AttributeError

print(my_tuple)  # Output: (1, 2, 3)

# Tuples can be used as dictionary keys

my_dict = { (1, 2): "Tuple Key" }
```

```
print(my_dict[(1, 2)])  # Output: Tuple Key
```

92) What is PEP8?

PEP8 consists of coding guidelines for Python language so that programmers can write readable code making it easy to use for any other person, later on.

93) Is all the memory freed when Python exits?

No it is not, because the objects that are referenced from global namespaces of Python modules are not always de-allocated when Python exits.

94) What does _init_.py do?

init.py is an empty py file used for importing a module in a directory. _init_.py provides an easy way to organize the files. If there is a module maindir/subdir/module.py,_init_.py is placed in all the directories so that the module can be imported using the following command-

```
import  maindir.subdir.module.
```

Functions of __init__.py

1. Marks a directory as a Python package.
2. Can execute initialization code when the package is imported.
3. Can define which modules should be accessible when the package is imported.

95) What is __init__() in Python?

The __init__() method is known as a constructor in object-oriented programming (OOP). It is used to initialize an object's attributes when an instance of a class is created. This method is automatically called upon object instantiation.

Purpose of __init__()

1. Assign values to object properties.
2. Perform any initialization tasks when an object is created.
3. Ensure objects start with a well-defined state.

Example

In this example, we create a Car class that initializes attributes like brand, model, and year using the __init__() constructor. The display_info() method prints the car details.

```
class Car:

    # Constructor

    def __init__(self, brand, model, year):

        self.brand = brand

        self.model = model

        self.year = year

    # Method to display car details

    def display_info(self):

        print(f"Car Details: {self.year} {self.brand} {self.model}")

# Creating instances of the Car class

car1 = Car("Toyota", "Corolla", 2022)

car2 = Car("Tesla", "Model S", 2023)

# Calling the method to display car details

car1.display_info()  # Output: Car Details: 2022 Toyota Corolla

car2.display_info()  # Output: Car Details: 2023 Tesla Model S
```

96) What is the different between range () and xrange () functions in Python?

range () returns a list whereas xrange () returns an object that acts like an iterator for generating numbers on demand.

97) What is monkey patching in Python?

Monkey patching is a technique that helps the programmer to modify or extend other code at runtime. Monkey patching comes handy in testing but it is not a good practice to use it in production environment as debugging the code could become difficult.

98) How are arguments passed in Python- by reference or by value?

The answer to this question is neither of these because passing semantics in Python are completely different. In all cases, Python passes arguments by value where all values are references to objects.

99) Explain the usage of decorators.

Decorators in Python are used to modify or inject code in functions or classes. Using decorators, you can wrap a class or function method call so that a piece of code can be executed before or after the execution of the original code. Decorators can be used to check for permissions, modify or track the arguments passed to a method, logging the calls to a specific method, etc.

100) What do you mean by list comprehension?

The process of creating a list while performing some operation on the data so that it can be accessed using an iterator is referred to as List Comprehension.

Example:

[ord (j) for j in string.ascii_uppercase]

[65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90]

101) What do you mean by pickling and unpickling in Python?

Python has a module called pickle which accepts any python object as an input and transforms it into a string representation before dumping it into a file using the dump function. This process is called pickling. The process of obtaining python objects from a pickled file is called unpickling.

Python Interview Questions for FAANG Companies

Below are some commonly asked Python interview questions in companies like Facebook (Meta), Amazon, Apple, Netflix, and Google. While the exact questions depend on the role, these will give you a solid starting point.

Facebook/Meta Python Interview Questions

102) How do you determine the maximum profit from a stock trade?

Given a list of stock prices in chronological order, the goal is to buy at a low price and sell at a high price to maximize profit. If making a profit is not possible, the loss should be minimized.

Example

```
stock_prices = [8, 4, 12, 9, 20, 1]

# Best buy price = 4, best sell price = 20 → Max profit = 16

stock_prices = [8, 6, 5, 4, 3, 2, 1]

# Best buy price = 6, best sell price = 5 → Minimized loss = -1
```

Solution Approach

1. Initialize variables to track the lowest buying price and maximum profit.
2. Iterate through the list, updating the lowest price seen so far.
3. Calculate potential profit and update the maximum profit if a better one is found.

Python Implementation

```
def max_stock_profit(prices):
    min_price = prices[0]
    max_profit = 0
    for price in prices[1:]:
        profit = price - min_price
```

```
        max_profit = max(max_profit, profit)

        min_price = min(min_price, price)

    return max_profit

# Example Usage

print(max_stock_profit([10, 9, 16, 17, 19, 23]))  # Output: 14

print(max_stock_profit([8, 6, 5, 4, 3, 2, 1]))  # Output: 0
```

This O(n) solution efficiently finds the best buying and selling points.

Amazon Python Interview Questions

103) How do you check if an array contains a Pythagorean triplet?

A Pythagorean triplet consists of three numbers (a, b, c) where $a^2 + b^2 = c^2$.

Example

Input: [3, 1, 4, 6, 5] → Output: True

Input: [10, 4, 6, 12, 5] → Output: False

Solution Approach

1. Square all elements and sort them.
2. Use two-pointer search to check if any two numbers sum up to another squared value.

Python Implementation

```
def has_pythagorean_triplet(arr):

    squared = sorted([x ** 2 for x in arr])

    for i in range(len(squared) - 1, 1, -1):

        left, right = 0, i - 1

        while left < right:
```

```
        if squared[left] + squared[right] == squared[i]:
            return True
        elif squared[left] + squared[right] < squared[i]:
            left += 1
        else:
            right -= 1
    return False
# Example Usage
print(has_pythagorean_triplet([3, 2, 4, 6, 5]))  # Output: True
```

This method runs in $O(n^2)$ time complexity.

104) How do you calculate the number of ways to make change for a given amount?

Given a list of coin denominations and a target amount, determine how many different ways the amount can be formed using those denominations.

Example

Denominations = [1, 2, 5], Amount = 5 → Output: 4 ways

(5, 2+2+1, 2+1+1+1, 1+1+1+1+1)

Solution Approach

1. Use Dynamic Programming to store solutions for smaller subproblems.
2. Iterate through coin denominations and update the number of ways to form each amount.

Python Implementation

```
def coin_change_ways(denominations, amount):
    ways = [0] * (amount + 1)
```

```
    ways[0] = 1

    for coin in denominations:

        for i in range(coin, amount + 1):

            ways[i] += ways[i - coin]

    return ways[amount]

# Example Usage

print(coin_change_ways([1, 2, 5], 5))  # Output: 4
```

This O(n × m) solution efficiently solves the problem.

Google Python Interview Questions

105). Explain lambda functions, iterators, and generators in Python.

- Lambda Function: An anonymous function defined using lambda, useful for short operations.
- Iterator: An object that allows sequential access to elements using __iter__() and __next__().
- Generator: A function that yields values lazily using yield, improving memory efficiency.

Example

```
# Lambda Function

square = lambda x: x * x

print(square(4))  # Output: 16

# Iterator

class Counter:

    def __init__(self, start, end):

        self.current = start
```

```
        self.end = end
    def __iter__(self):
        return self
    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        self.current += 1
        return self.current - 1
counter = Counter(1, 5)
for num in counter:
    print(num)  # Output: 1, 2, 3, 4, 5
# Generator
def countdown(n):
    while n > 0:
        yield n
        n -= 1
for val in countdown(3):
    print(val)  # Output: 3, 2, 1
```

106) How do you find the maximum difference j - i where arr[j] > arr[i]?

Example

Input: [20, 70, 40, 50, 12, 38, 98] → Output: 6 (j = 6, i = 0)

Solution Approach

1. Maintain minimum index from the left while iterating.
2. Update the max difference when a larger element is found.

Python Implementation

```
def max_index_diff(arr):
    min_index = 0
    max_diff = 0
    for j in range(1, len(arr)):
        if arr[j] > arr[min_index]:
            max_diff = max(max_diff, j - min_index)
        else:
            min_index = j
    return max_diff
# Example Usage
print(max_index_diff([20, 70, 40, 50, 12, 38, 98]))  # Output: 6
```

This approach runs in O(n) time.

107) How do you implement the ternary operator in Python?

A ternary operator is a compact way to write conditional expressions.

Example

```
# Traditional if-else
score = 75
if score < 50:
    print("Fail")
```

```
elif score < 70:
    print("Merit")
else:
    print("Distinction")
# Equivalent ternary
print("Fail" if score < 50 else "Merit" if score < 70 else "Distinction")
```

108) How do you implement an LRU Cache in Python?

Python provides functools.lru_cache() for caching function calls.

Example

```
from functools import lru_cache
@lru_cache(maxsize=3)
def add(a, b):
    return a + b
print(add(1, 2))  # Computed
print(add(1, 2))  # Cached
```

Alternatively, use OrderedDict from collections for manual implementation.

Explore Python Data Science Projects Ideas with ProjectPro!

Succeeding in a data science interview isn't just about theoretical knowledge—it's about showcasing real-world problem-solving skills through hands-on projects. With ProjectPro, you gain access to 250+ solved end-to-end Data Science, Machine Learning, and Generative AI projects, helping you build a standout portfolio that proves your expertise.

Additionally, ProjectPro's AI-powered interview coach prepares you for both technical and behavioral questions, providing real-time feedback to refine your answers. Practice anytime,

boost your confidence, and get job-ready with guided projects and expert mentorship—all in a flexible, stress-free environment!

FAQs

1. **What are the basic Python questions asked in an interview?**

Basic Python interview questions often focus on data types, loops, functions, list comprehensions, string manipulation, and exception handling. Common topics include difference between lists and tuples, mutable vs. immutable types, lambda functions, and Python's memory management. These questions assess fundamental Python knowledge and coding proficiency.

2. **How do I prepare for a Python interview?**

To prepare for a Python interview, focus on core concepts, problem-solving, and hands-on coding practice. Review data structures, algorithms, OOP, file handling, and libraries like NumPy and Pandas. Solve problems on LeetCode, HackerRank, and work on real-world projects to strengthen practical skills. Mock interviews can also help.

3. **What is listed in Python interview questions?**

Python interview questions cover basic syntax, object-oriented programming, data handling, file operations, and advanced topics like decorators, generators, and multithreading. They may also include data science-related topics like Pandas, NumPy, and machine learning libraries if the role requires them. Coding challenges often test logic, efficiency, and problem-solving skills.